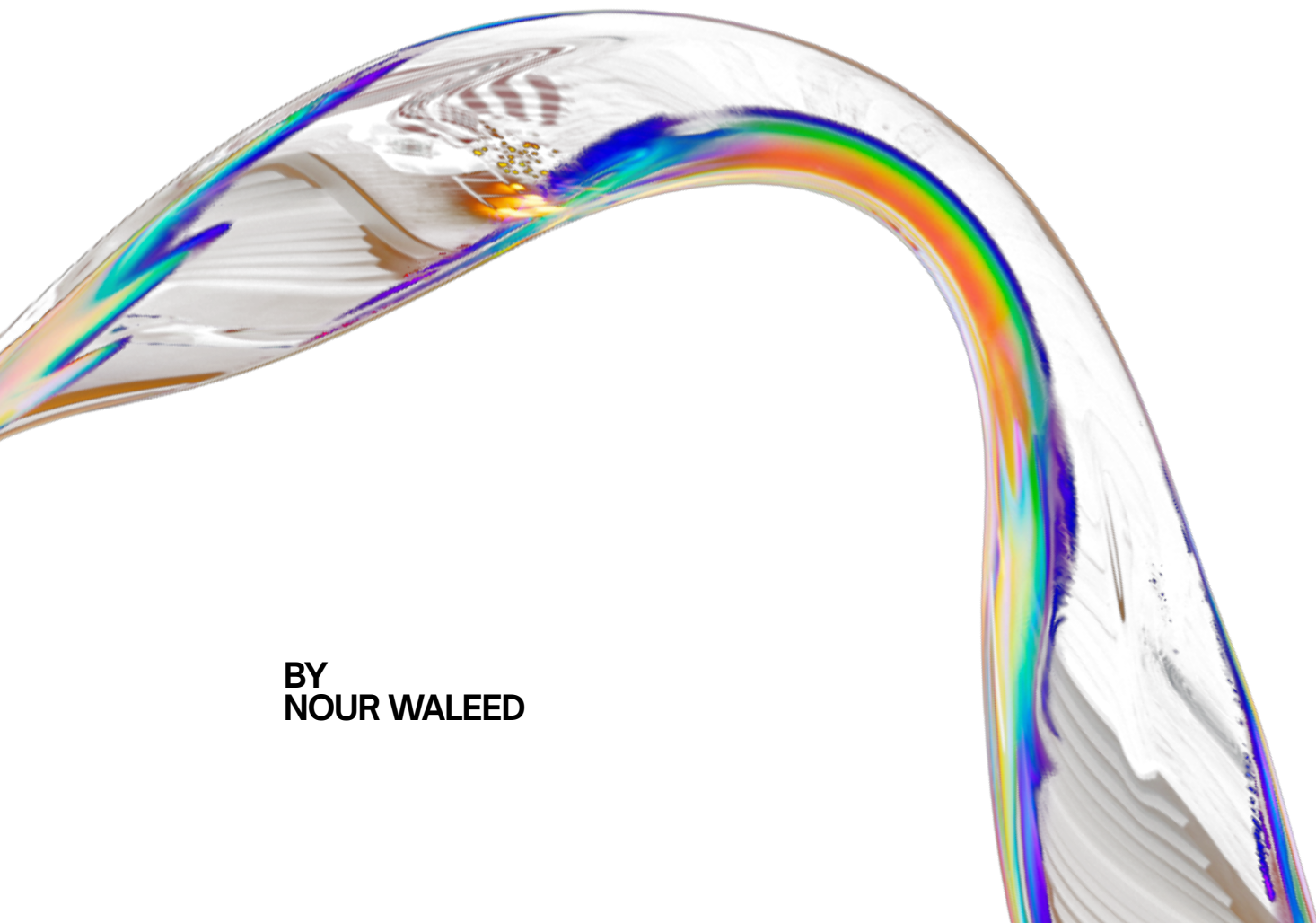


Background Subtraction

Digital Image Processing
Project



BY
NOUR WALEED



TODAY'S AGENDA



INTRODUCTION

OBJECTIVES

DATASET

CHALLENGES



LIBRARIES

METHODOLOGY

PIPELINE

RESULTS

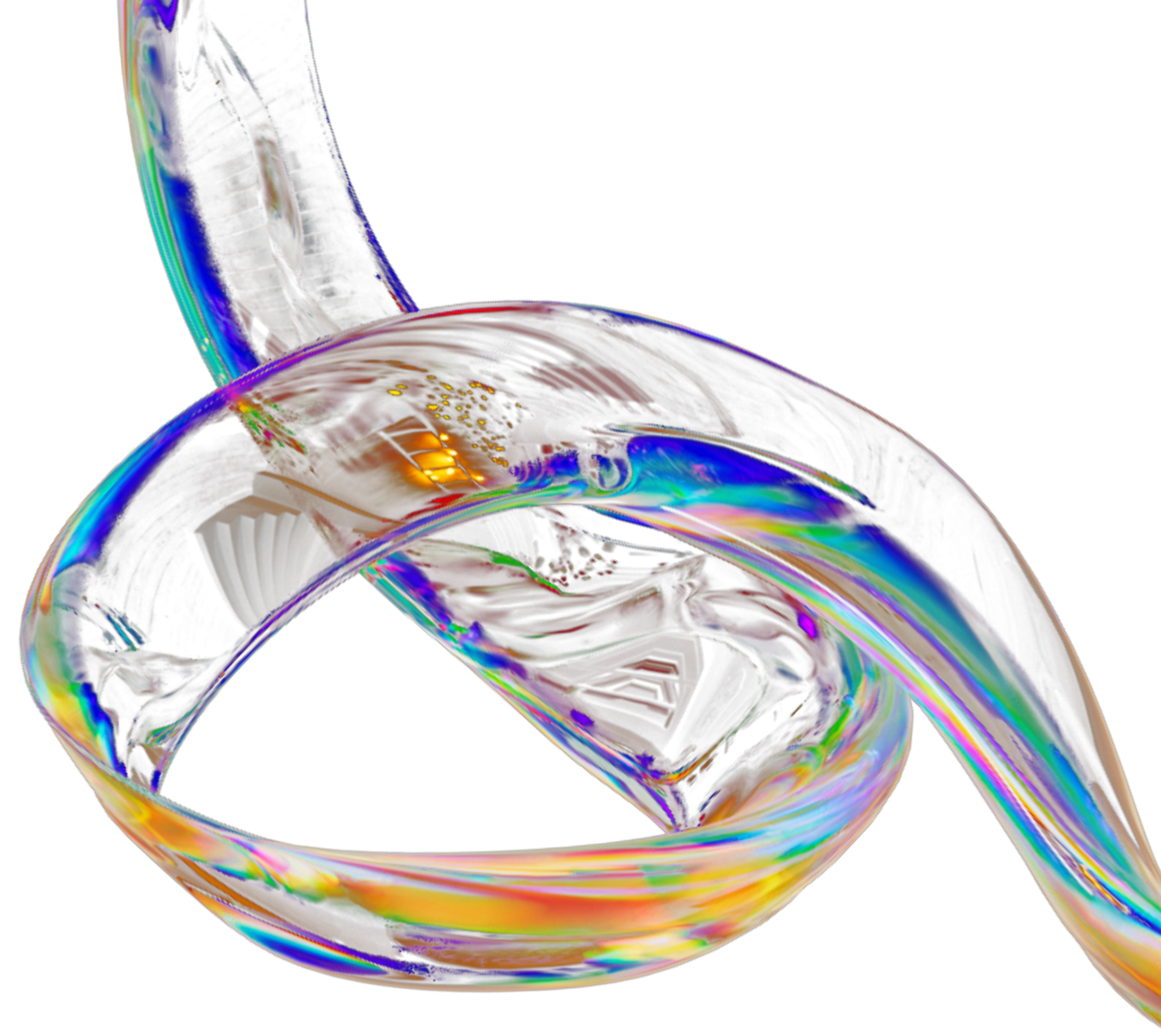
Introduction

What is BG Subtraction?

Background subtraction is a classical computer-vision technique used to identify moving objects in a scene by separating the foreground (motion) from the background (static parts).

Applications

- Surveillance
- Traffic monitoring
- Pedestrian tracking
- Industrial automation
- Sports analysis

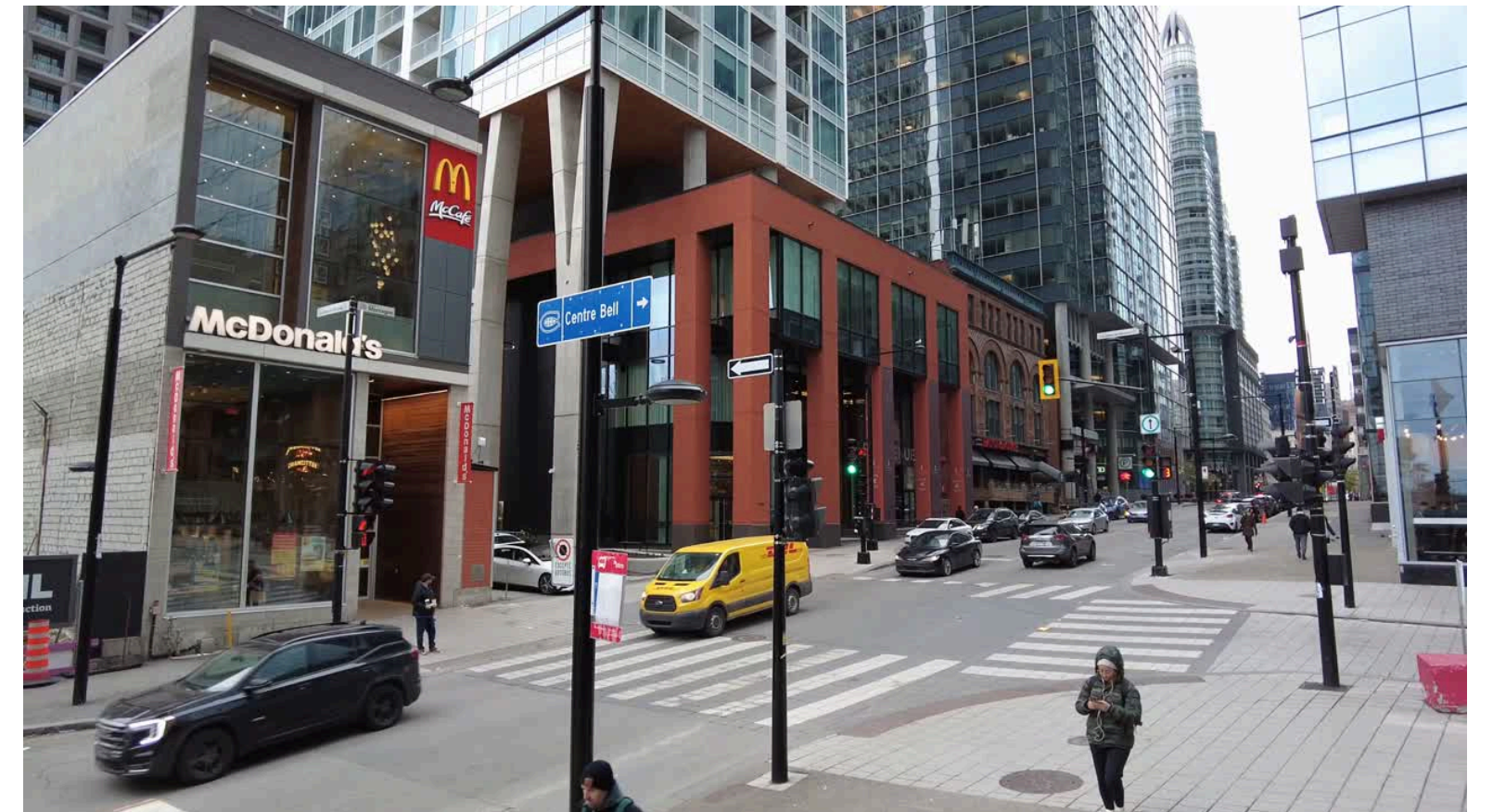


Objectives

- Detect moving objects in a real urban scene
- Use classical background subtraction (not deep learning)
- Handle 4K video by downscaling to manageable resolution
- Remove noise caused by reflections, building textures and camera movement
- Extract contours of moving objects
- Draw bounding boxes around valid detections

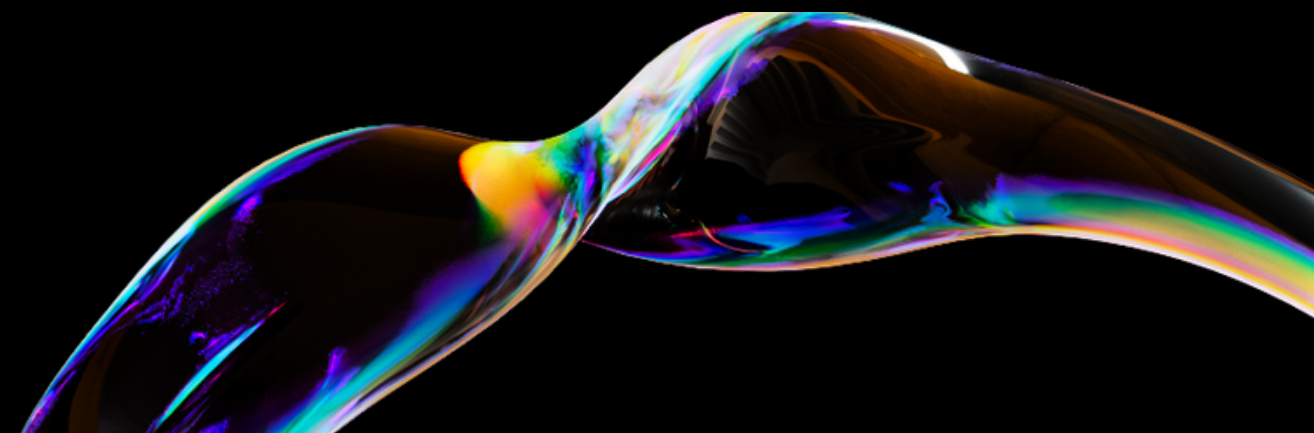
Dataset

- **Input Video: 4K UHD (3840 × 2160) urban street scene**
- **Contains pedestrians, moving vehicles and background motion artifacts**



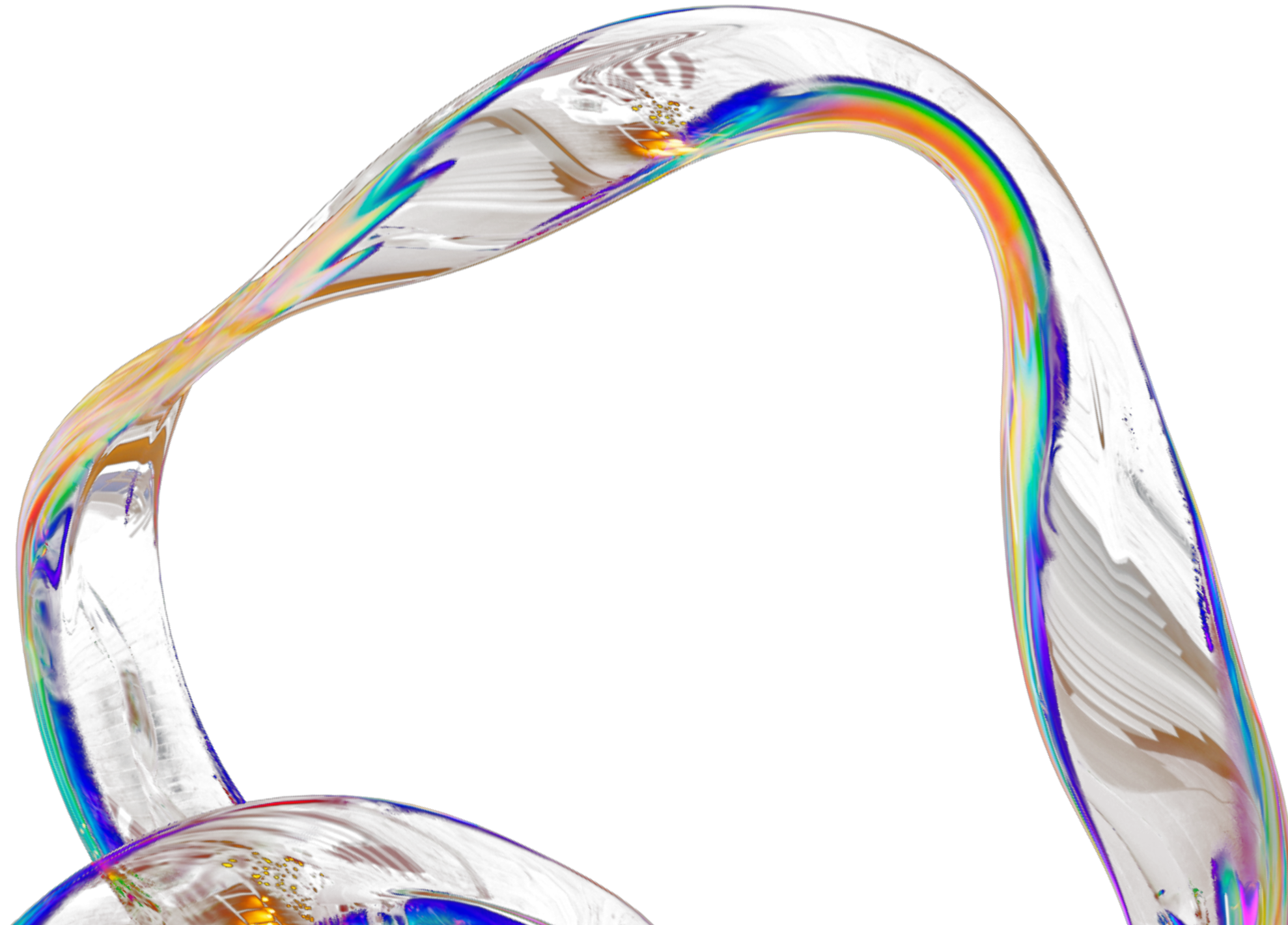
Challenges

- 4K video created excessive noise and overwhelmed MOG2
- Glass reflections from building produced false motion
- Distant small objects had too little pixel motion
- MOG2 required warm-up time to learn background
- High-resolution textures triggered false contours



Libraries and Tools

- Python
- OpenCV (cv2)
- NumPy
- Google Colab



METHODOLOGY OVERVIEW

- **PREPROCESSING (RESIZE, ROI, BLUR)**
- **BACKGROUND MODELING (MOG2)**
- **BINARY MOTION MASK GENERATION (THRESHOLDING)**
- **NOISE REDUCTION (MORPHOLOGY)**
- **OBJECT EXTRACTION (CONTOURS)**
- **OBJECT LOCALIZATION (BOUNDING BOXES)**



Pipeline

- Cell 1
- Imports

```
import cv2 as cv
import numpy as np
from google.colab.patches import cv2_imshow
print("OpenCV Version:", cv.__version__)
```

```
• OpenCV Version: 4.12.0
```

Pipeline

```
cap = cv.VideoCapture("/content/19613728-uhd_3840_2160_60fps.mp4")
if not cap.isOpened():
    print("Error: Could not open video.")
else:
    ret, frame = cap.read()

    if not ret:
        print("Error: Could not read frame.")
    else:
        height, width, channels = frame.shape
        print("Frame size:", height, "x", width)
        cv2.imshow(frame)
cap.release()
```

- Cell 2
- Video loading and preview

Pipeline

- Cell 3
- **Background Subtractor (MOG2) Setup + Resize Constants**

```
FRAME_WIDTH  = 640
FRAME_HEIGHT = 360

history = 500
varThreshold = 50
detectShadows = False

backSub = cv.createBackgroundSubtractorMOG2(
    history=history,
    varThreshold=varThreshold,
    detectShadows=detectShadows
)

print("MOG2 model created with:")
print("History:", history)
print("varThreshold:", varThreshold)
print("detectShadows:", detectShadows)
print("Processing size:", FRAME_HEIGHT, "x", FRAME_WIDTH)
```

Pipeline

- Cell 4
- Background Subtraction Test (resized full frame)

```
cap = cv.VideoCapture("/content/19613728-uhd_3840_2160_60fps.mp4")

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    backSub = cv.createBackgroundSubtractorMOG2(
        history=500,
        varThreshold=50,
        detectShadows=False
    )

    warmup_frames = 80
    for i in range(warmup_frames):
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
        blurred = cv.GaussianBlur(frame, (5,5), 0)
        backSub.apply(blurred)

    frame_count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
        roi = frame[150:360, :]
        frame = roi
        blurred = cv.GaussianBlur(frame, (5,5), 0)

        fgmask = backSub.apply(blurred)
        _, fgmask = cv.threshold(fgmask, 200, 255, cv.THRESH_BINARY)

        cv2_imshow(fgmask)
```

Pipeline

```
cap = cv.VideoCapture("/content/19613728-uhd_3840_2160_60fps.mp4")

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    backSub = cv.createBackgroundSubtractorMOG2(
        history=500,
        varThreshold=50,
        detectShadows=False
    )

    warmup_frames = 80
    for i in range(warmup_frames):
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
        blurred = cv.GaussianBlur(frame, (5,5), 0)
        backSub.apply(blurred)

    kernel = np.ones((5,5), np.uint8)
    frame_count = 0
```

- Cell 5
- Morphological Filtering

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
    blurred = cv.GaussianBlur(frame, (5,5), 0)

    fgmask = backSub.apply(blurred)
    _, fgmask = cv.threshold(fgmask, 200, 255, cv.THRESH_BINARY)

    opening = cv.morphologyEx(fgmask, cv.MORPH_OPEN, kernel)
    clean_mask = cv.morphologyEx(opening, cv.MORPH_CLOSE, kernel)

    cv2_imshow(clean_mask)

    frame_count += 1
    if frame_count >= 30:
        break

cap.release()
```

Pipeline

- Cell 6
- Final Detection: Contours + Bounding Boxes

```
VIDEO_PATH = "/content/19613728-uhd_3840_2160_60fps.mp4"
FRAME_WIDTH = 640
FRAME_HEIGHT = 360

backSub = cv.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=50,
    detectShadows=False
)

cap = cv.VideoCapture(VIDEO_PATH)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    warmup_frames = 80
    for i in range(warmup_frames):
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
        roi = frame[150:FRAME_HEIGHT, :]
        blurred = cv.GaussianBlur(roi, (5,5), 0)
        backSub.apply(blurred)

    kernel = np.ones((5,5), np.uint8)
    frame_count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

Pipeline

- Cell 6
- Final Detection: Contours + Bounding Boxes

```
frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
roi   = frame[150:FRAME_HEIGHT, :]

blurred = cv.GaussianBlur(roi, (5,5), 0)
fgmask  = backSub.apply(blurred)
_, fgmask = cv.threshold(fgmask, 200, 255, cv.THRESH_BINARY)

opening  = cv.morphologyEx(fgmask, cv.MORPH_OPEN, kernel)
clean_mask = cv.morphologyEx(opening, cv.MORPH_CLOSE, kernel)
dilated  = cv.dilate(clean_mask, kernel, iterations=2)

contours, _ = cv.findContours(dilated, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    area = cv.contourArea(cnt)
    if area < 385:
        continue

    x, y, w, h = cv.boundingRect(cnt)

    pad = 5
    x = max(0, x - pad)
    y = max(0, y - pad)
    w = w + 2*pad
    h = h + 2*pad

    cv.rectangle(roi, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2_imshow(roi)

frame_count += 1
if frame_count >= 30:
    break

cap.release()
```

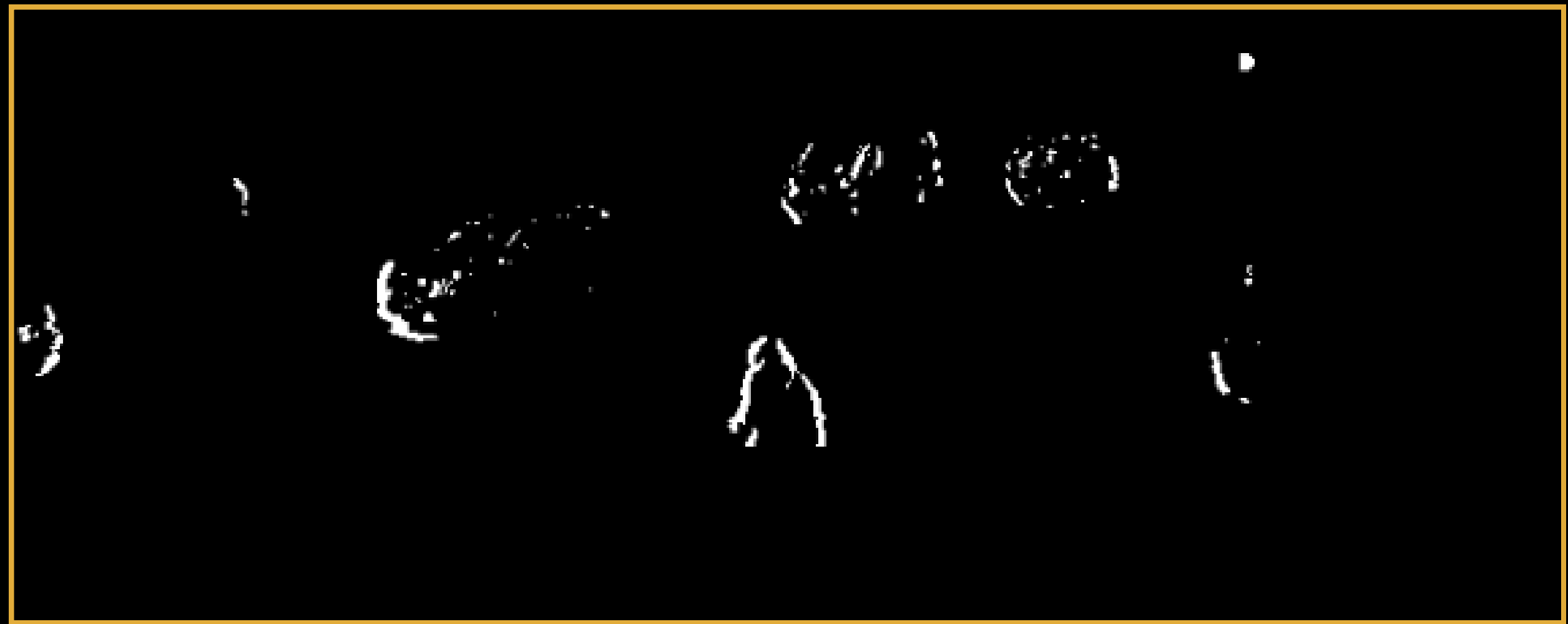
RESULTS

- ORIGINAL INPUT FRAME



RESULTS

- RAW BACKGROUND SUBTRACTION OUTPUT



RESULTS

- MORPHOLOGICAL CLEANING RESULTS
- RAW FOREGROUND MASK BEFORE MORPHOLOGICAL CLEANING



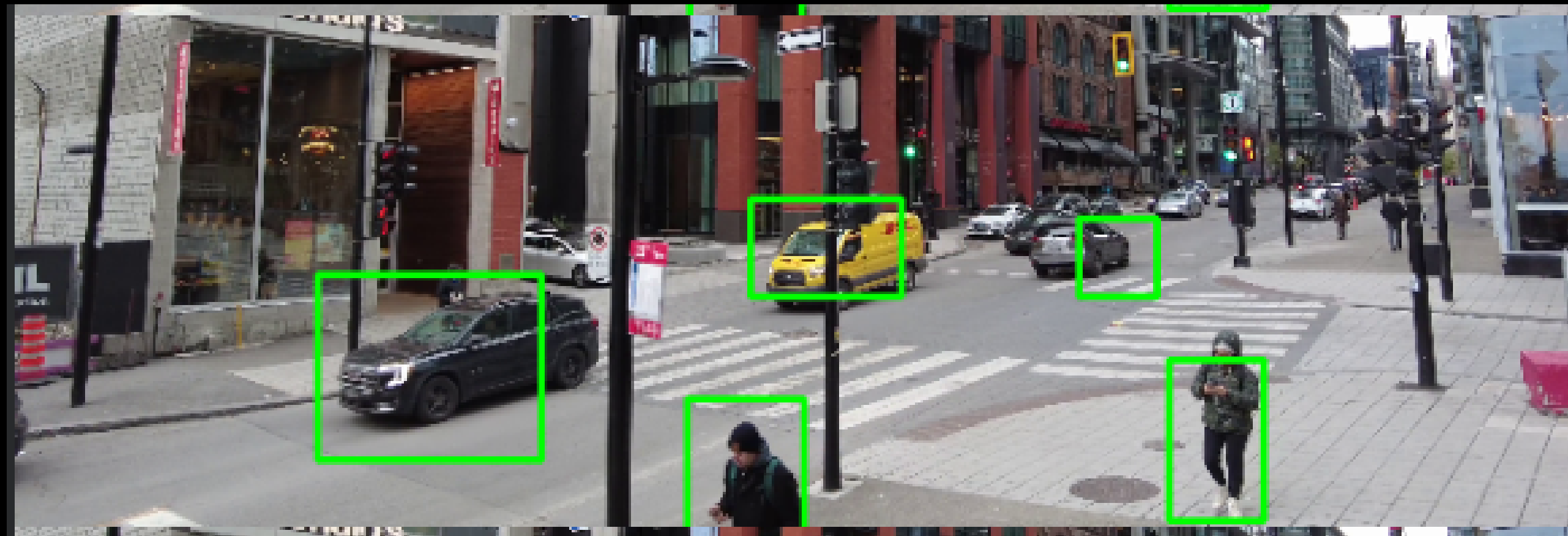
RESULTS

- MORPHOLOGICAL CLEANING RESULTS
- FOREGROUND MASK AFTER MORPHOLOGY (OPENING + CLOSING + DILATION)



RESULTS

- FINAL DETECTION OBJECTS WITH BOUNDING BOXES





Thank You

BY
NOUR WALEED